

Revisión del modelo de datos

Entrega revisada: diagrama entidad-relación y script DDL del equipo de practicantes,
27 de agosto de 2026.

Contrastado con `BACKEND-BRIEF.md` y las interfaces vigentes en `src/data/`.

4 CAMBIOS OBLIGATORIOS

4 DECISIONES A CONFIRMAR

PRIVACIDAD RESPETADA

El modelo cubre las siete entidades pedidas en el brief y está bien razonado.
Hay cuatro correcciones que deben hacerse antes de empezar a programar
y cuatro decisiones que no les corresponde tomar solos: hay que
confirmarlas con la Alcaldía o con el equipo de frontend.

01 Lo que está bien

- **Privacidad respetada:** ninguna tabla almacena datos de usuarias. Es el punto más importante del encargo y está resuelto. Debe seguir así.
- `phones` y `workingDates` normalizados en tablas aparte en lugar de una columna JSON, con la justificación escrita: permite CRUD por número.
- `admin_users` con `hashed_password` y sin endpoint público de registro.
- Uso de tipos `time` y `date` reales en vez de cadenas de texto, como estaban en el frontend.
- Cada tabla lleva un comentario que explica la decisión de diseño. Muy buena práctica; mantenerla.

2.1 `category` no puede ser un `ENUM` de SQL **OBLIGATORIO**

El brief lo pide de forma explícita (§5.1): la taxonomía actual es brasileña (CEAM, NEAM...) y debe reemplazarse por la colombiana (Comisaría de Familia, Casa de Justicia, Centro de Salud...). Con un `ENUM`, cada cambio exige una migración; con una tabla catálogo la Alcaldía lo edita desde la API.

Ojo también con los nombres: hoy `equipment_types` es la taxonomía *fin*a y `category` la *grues*a, lo que se lee al revés.

QUÉ HACER

Crear `equipment_categories` (`id`, `slug`, `label`, `orden`) y usar `category_id` como llave foránea. Sugerencia de nombres: `equipment_categories` para la taxonomía gruesa (= `FilterTag`) y `unit_types` para la fina (= `DetailedFilterTag`).

2.2 El SQL mezcla dialectos y no corre en ningún motor **OBLIGATORIO**

`AUTO_INCREMENT` y `ALTER TABLE ... COMMENT =` son de MySQL; `timestampz` es de PostgreSQL. Sirve como diagrama, no como esquema ejecutable.

QUÉ HACER

El entregable no es DDL escrito a mano, sino los modelos SQLAlchemy en `app/models/` más la primera migración de Alembic: PostgreSQL en producción y SQLite en local, según el brief §3.

2.3 Faltan las cláusulas `ON DELETE` en todas las llaves foráneas **OBLIGATORIO**

Al borrar un equipamiento quedan huérfanos en `equipment_phones`, `equipment_working_dates` y `equipment_violence_types`, y el `DELETE` falla.

QUÉ HACER

`ON DELETE CASCADE` en esas tres tablas hijas y `ON DELETE RESTRICT` (o `SET NULL`) en `equipments.equipment_type_id` y `referrals.violence_type_id`. Falta además el índice único en `(equipment_id, phone_number)`; sólo pusieron en `(equipment_id, work_date)`: mismo criterio.

2.4 `Equipment.violenceTypes` hoy no es una relación OBLIGATORIO

En el frontend es un texto libre ("`violências física; psicológica, moral...`"), no una lista de identificadores. Modelarlo como N:M contra el glosario educativo es mejor diseño, pero cambia el contrato con el frontend y rompe el seed, porque habría que interpretar texto libre.

ELEGIR UNA OPCIÓN Y DEJARLA ESCRITA

A. Mantener `violence_types: str?` — migración transparente, sin cambios en el frontend. **B.** Hacer el N:M — mejor modelo, pero exige avisar al equipo de Next.js y definir cómo se convierten los datos actuales.

03 Revisar los datos reales antes del seed

Van a chocar con esto en el Sprint 2:

- `phones` usa el número como identificador (`id: 190` , `id: 192`).
- `violenceTypes` empieza en `id: 0` .
- Con `AUTO_INCREMENT` o `serial` hay que insertar los identificadores explícitos en el seed y luego ajustar la secuencia (`setval` en PostgreSQL), o aceptar que los identificadores cambian y comunicarlo al frontend.

04 Decisiones a confirmar

Estas cuatro no las tomen solos: se resuelven con la Alcaldía o con el equipo de frontend.

TEMA	PREGUNTA
<code>referrals</code>	¿Un mismo referral puede aplicar a varios tipos de violencia? En los datos actuales hay uno llamado «Violencia Moral y Psicológica» con un único <code>idTipoViolencia</code> . Si aplica a varios, la llave foránea obligatoria 1:N no alcanza.
<code>equipment_working_dates</code>	¿Son fechas en que la unidad sí abre, como excepción al horario, o fechas en que cierra? Definirlo antes de programar el cálculo de «abierto ahora» junto con <code>holidays</code> .
<code>referrals.text</code>	¿Texto plano, Markdown o HTML? Si es HTML hay que sanitizarlo al guardarlo, porque viene del panel de administración.
Categorías	¿La API devuelve el slug (<code>salud</code>) o la etiqueta que ve la usuaria («Unidad de Salud», «Servicios especializados para la Mujer», «Estación de Policía»)? Por eso el catálogo necesita <code>slug</code> y <code>label</code> .

05 Detalles menores

- `created_at` y `updated_at` están solo en `equipments`: ponerlos en todas las tablas o en ninguna. Para la Alcaldía sería útil además un `updated_by`.
- Considerar `is_active` en `equipments` en lugar de borrado físico: una unidad que cierra temporalmente no debería perder sus datos.
- El índice compuesto en `(latitude, longitude)` casi no ayuda a las consultas por *bbox*. Con unas decenas de unidades en Marinilla no hace falta ningún índice; si más adelante hiciera falta, la solución es PostGIS. No optimizar todavía.
- `open_24h` pasó de `0|1` en el frontend a `boolean`: está bien, pero anotararlo en la tabla de equivalencias.

06 Próximo entregable

1. **Tabla de equivalencias** entre el tipo de TypeScript, la columna de la base de datos y el campo de la API, con los alias camelCase de Pydantic. Es lo que evita sorpresas en la integración.
2. **Sprint 1 del brief:** FastAPI corriendo, `GET /health`, Alembic configurado y el CRUD completo de `Phone` de punta a punta con pruebas. Con eso validamos el patrón antes de replicarlo en las seis entidades restantes.